Features

În ordinea în care se construiesc · bifezi când merge pe EC2 · Rareș

1 Frizeri

administrator · vizitator

☐	F1.1	Administratorul adaugă un frizer: cont (email, parolă) + profil (nume, prenume, telefon).	`createBarber`
☐	F1.2	Administratorul modifică profilul unui frizer.	`updateBarber`
☐	F1.3	Administratorul retrage un frizer: nu mai apare și nu mai primește programări.	`deactivateBarber`
☐	F1.4	Oricine vede frizerii activi și profilul unuia.	`getBarbers` · `getBarber`

Reguli

- Nume, prenume, telefon obligatorii; email valid și unic (400 / 409); parolă de minim 8 caractere. Contul primește rolul `BARBER`; contul și profilul se salvează împreună sau deloc.
- Frizer inexistent → 404. De la feature-ul 6: frizerul cu programări viitoare se retrage doar după ce i le anulezi (409).
- Până la feature-ul 3 scrierea e deschisă tuturor; de la feature-ul 3, doar `ADMIN`.

2 Procedurile frizerului

administrator · frizer · vizitator

☐	F2.1	Se adaugă o procedură la un frizer: nume, descriere, durată în minute, prețul lui.	`createBarberProcedure`
☐	F2.2	Se modifică o procedură a frizerului.	`updateBarberProcedure`
☐	F2.3	Se retrage o procedură: nu mai apare, dar programările vechi o păstrează.	`deactivateBarberProcedure`
☐	F2.4	Oricine vede procedurile active ale unui frizer, cu durată și preț.	`getBarberProcedures`

Reguli

- Nume obligatoriu, maxim 100 de caractere, unic la același frizer (400 / 409) — alt frizer poate avea același nume, cu alt preț. Durata > 0; prețul ≥ 0 (400).
- Procedura altui frizer, pe ruta acestui frizer → 404. De la feature-ul 3 scrie `ADMIN` sau frizerul însuși, doar la el (403).

3 Conturi și clienți

vizitator · orice utilizator · administrator

☐	F3.1	Vizitatorul își face cont de client: email, parolă, nume, prenume, telefon.	`register`
☐	F3.2	Utilizatorul se autentifică cu email și parolă și primește un token valabil o oră.	`login`
☐	F3.3	Utilizatorul autentificat își vede datele contului.	`getCurrentUser`
☐	F3.4	Administratorul vede lista clienților.	`getCustomers`
☐	F3.5	Administratorul blochează sau deblochează un client.	`setCustomerBlocked`

Reguli

- Email valid și unic, telefon unic (409); parolă de minim 8 caractere; nume și prenume obligatorii.
- Prin înregistrare se face doar cont `CUSTOMER`. Contul `ADMIN` îl creează aplicația la pornire, din `ADMIN_EMAIL` și `ADMIN_PASSWORD`, dacă nu există.
- La login greșit, același mesaj pentru email inexistent și pentru parolă greșită (401). Parola nu apare în niciun răspuns; în bază stă doar hash-ul ei.

4 Program și concedii

administrator · frizer · vizitator

☐	F4.1	Se setează programul pe săptămână al unui frizer: pentru fiecare zi lucrată, ora de început și de sfârșit.	`setBarberSchedule`
☐	F4.2	Oricine vede programul unui frizer.	`getBarberSchedule`
☐	F4.3	Se adaugă o perioadă liberă (concediu, pauză), cu motiv opțional.	`createBarberTimeOff`
☐	F4.4	Se văd și se șterg perioadele libere ale unui frizer.	`getBarberTimeOffs` · `deleteBarberTimeOff`

Reguli

- Scrie `ADMIN` sau frizerul însuși, doar la el (403).
- Programul se înlocuiește tot odată; zi repetată în aceeași cerere → 400; sfârșit ≤ început → 400.
- Perioadă liberă peste o programare existentă → 409.

5 Ore libere

vizitator

☐	F5.1	Oricine alege un frizer, o zi și una sau mai multe proceduri ale lui și vede orele la care poate începe programarea.	`getAvailability`

Reguli

- Procedurile sunt ale frizerului ales și active (400). Durata = suma duratelor lor. O oră e liberă dacă tot intervalul intră în programul zilei și nu atinge o perioadă liberă sau o programare ne-anulată.
- Orele se propun din 15 în 15 minute; în ziua de azi, doar cele din viitor.

6 Programări

client · frizer · administrator

☐	F6.1	Clientul se programează la un frizer, la o oră, pentru una sau mai multe proceduri, cu o notă opțională.	`createAppointment`
☐	F6.2	Clientul își vede programările, ordonate după oră.	`getMyAppointments`
☐	F6.3	Frizerul (sau administratorul) vede programările unui frizer pe o perioadă.	`getBarberAppointments`
☐	F6.4	Oricine are legătură cu programarea o vede în detaliu, cu procedurile și prețul total.	`getAppointment`
☐	F6.5	Clientul, frizerul sau administratorul anulează o programare.	`cancelAppointment`
☐	F6.6	Frizerul confirmă programarea, apoi o marchează efectuată.	`updateAppointmentStatus`

Reguli

- Doar un client neblocat se programează (403); ora e în viitor (400); frizerul există și e activ (404); procedurile sunt ale lui și active (400).
- Ora trebuie să fie una dintre cele din F5.1; altfel 409 — și când doi clienți cer aceeași oră în aceeași secundă.
- Prețul și durata se copiază din procedura frizerului în `appointment_procedures`.
- Statusul merge doar înainte: `BOOKED` → `CONFIRMED` → `COMPLETED`; anularea doar din `BOOKED` / `CONFIRMED`; altfel 409.
- Clientul vede și anulează doar programările lui, frizerul doar pe ale lui (403).

7 Programări recurente

client

☐	F7.1	Clientul cere o programare care se repetă: frizer, proceduri, prima zi, ora, din câte în câte săptămâni, până când.	`createRecurringAppointment`
☐	F7.2	Clientul își vede seriile de programări.	`getMyRecurringAppointments`
☐	F7.3	Clientul oprește o serie: programările ei viitoare se anulează.	`cancelRecurringAppointment`

Reguli

- Se generează toate programările seriei, fiecare după regulile din F6.1. Dacă una singură nu încape → 409 cu datele care nu încap, și nu se salvează nimic.
- Seria are cel mult 6 luni; interval între 1 și 8 săptămâni (400).